

Problem C:

The FloorSet Challenge: Data-Driven SoC

Floorplanning

*Uday Mallappa, Sachin Bhat, Pei Chun Ch'ng, Somdeb Majumdar,
Yuan Lu, Yusu Wang
(Intel)*

**To facilitate centralized issue tracking and faster response times,
you are encouraged to submit your questions directly through the Git repository.**

Q&A

Q1. The write-up mentions that any deviation in dimensions for fixed-shape and preplaced blocks is a hard-violation and should result in a score of 10, but then there is also a soft-constraint formula given to compute these violations, which are contradictory in nature. Could you please specify whether to consider this as a hard or soft constraint?

cost.py provided in the GitHub repo doesn't explicitly consider hard-constraint penalty ($M=10$) or the alpha, beta and gamma values given in the cost function. Does this mean that the organizers are yet to provide a proper validator program?

A1. Here is the contest repo, that addresses these concerns.

<https://github.com/IntelLabs/FloorSet/tree/main/iccad2026contest>

Q2. I am writing to request clarification regarding the technical environment and evaluation process for Problem C.

Specifically, I would like to inquire about the following details:

1. Memory Capacity: Could you please provide the specific memory (RAM/VRAM) limits allocated for each submission during evaluation?
2. Batch Processing: During the validation and test phases, can the test cases be processed in batches, or they should be evaluated sequentially?

A2. These are the hardware specs -> A100 80GB VRAM GPU, ICELAKE CPU with 48 cores, 128GB RAM

Batch processing -> The test cases will be evaluated sequentially.

Q3. I am writing to clarify the hardware specifications for the hidden test set evaluation environment, as this heavily impacts our inference strategy and the "Runtime Factor" score.

Could you please provide the following details regarding the evaluation server:

1. What specific CPU and GPU models will be used for evaluation?
2. What is the maximum RAM capacity allocated per submission?
3. Are we permitted to utilize multiprocessing or multithreading during inference?

A3.

1. A100 80GB VRAM GPU, ICELAKE CPU with 48 cores
2. 128GB RAM
3. The test samples are evaluated sequentially. But for each sample evaluation, multiprocessing or multithreading can be used.

Q4. We found that the training data golden answers violate MIB, Boundary, and Group constraints in some cases.

Since these are soft constraints, are the golden answers themselves penalized?

A4. Yes, seeing soft violations in training data is normal and consistent with how the dataset is intended to be used.

Q5. I am writing to ask about how to handle a potential constraint conflict in some test cases.

In particular, I found that in some examples, two modules may satisfy the following conditions at the same time:

They belong to the same boundary constraint.

They are both preplaced modules.

One of the preplaced modules does not touch the corresponding boundary.

In this case, if we strictly enforce both the preplaced-position constraint and the

boundary constraint, the problem appears to become infeasible. Could you please clarify how such cases are expected to be handled during evaluation?

Should we:

Prioritize keeping the preplaced modules fixed and allow the boundary constraint to be partially violated;

Prioritize satisfying the boundary constraint and adjust the preplaced module accordingly;

Treat the case as infeasible;

Or follow some other specified rule?

I would appreciate your clarification on the expected handling of such conflicting constraints so that we can make our implementation consistent with the evaluation rules.

A5. In the contest evaluator, preplaced position is a hard constraint, and boundary touching is a soft constraint. They operate at different levels and there is no ambiguity about priority.

Preplaced position (hard constraint): A preplaced block must appear at exactly the (x, y, w, h) specified in the input. Any deviation makes the solution infeasible and assigns $\text{Cost} = M = 10$ for that test case. This cannot be overridden by any soft constraint.

Boundary touching (soft constraint): Whether a block touches its required boundary edge or corner is checked as part of V_{soft} . A violation increments the boundary violation count, which raises the exponential penalty term $\exp(\beta \cdot V_{\text{relative}})$. The solution remains feasible.

The correct handling: Keep all preplaced modules at their fixed positions. If a preplaced module's fixed position does not touch its boundary constraint, that boundary violation is counted in V_{soft} and penalized through the cost function, but the solution is still feasible. You should never move a preplaced module to satisfy a soft constraint.

Q6. For the soft blocks, can they exceed their area target? If the area target is 100, can the block be 12x12? The reason I ask is that in order to get abutment between a pair of blocks, or to have a block touch the side or top of the placement area, it might be useful to stretch the block in one or more directions.

The contest specifies a 1% area tolerance, and going below the target area is clearly a violation, but it seems reasonable to allow things to be larger.

A6. No. The 1% area tolerance is enforced symmetrically in both directions using an absolute relative error check:

$$|w \times h - \text{area_target}| / \text{area_target} \leq 0.01$$

Both exceeding and falling short of the target by more than 1% count as a hard constraint violation, making the solution infeasible (Cost = M = 10). A block with target area 100 placed as 12×12 (area = 144, +44% excess) would be infeasible.

The rationale is that soft blocks have a target area reflecting the actual silicon area required by the module. Allowing unconstrained growth would let a solver trivially satisfy grouping, MIB, and boundary soft constraints by inflating block sizes, circumventing the intent of those constraints. The 1% tolerance exists only to accommodate minor aspect-ratio adjustments while preserving the overall die area budget.

If you need a block to touch a boundary or abut a neighbor, the correct approach is to adjust its aspect ratio (width/height) while keeping $w \times h$ within 1% of the target area, not to increase the area itself.

Q7. The submission guidelines state that participants should submit an optimizer executable (with PyInstaller recommended for Python submissions), while the provided evaluation script appears to import an optimizer.py file, instantiate the MyOptimizer class, and invoke its solve() method directly.

This raises some uncertainty about how executable-based submissions will be evaluated. Specifically, if participants submit a standalone binary executable instead of source code, it is unclear how the evaluation framework will interface with the optimizer, since the currently available evaluation script expects a Python module and class interface.

Could you please clarify:

1. Whether the final evaluation will still use the same Python evaluation script and import-based interface (optimizer.py with a MyOptimizer class and solve() method).
2. If executable submissions are supported, how the evaluation system will invoke the executable and exchange inputs/outputs with it.

3. Whether participants should prioritize compatibility with the provided evaluation script or package their solution solely as an executable.

The guidelines mention that evaluation will be performed "in the same command format provided in the evaluation script," which seems to suggest the Python interface will be used, but we would like to confirm this interpretation.

A7. We will wrap the participants' executable with a python script during evaluation. The focus for participants must be toward packaging their solution as an executable with all the dependencies.

Q8. How should the submitted executable receive input data (e.g., command-line arguments, stdin, or input files)?

A8. Adhere to same input schema of the solve() method in optimizer_template.py

Q9. How should the executable return the floorplanning results (e.g., stdout or output files), and what is the expected format?

A9. Adhere to same output schema of the solve() method in optimizer_template.py

Q10. Will the final evaluation still require compatibility with the MyOptimizer.solve() interface?

A10. Yes

Q11. Could you provide an executable interface specification or an example wrapper script so that participants can properly test their packaged solutions before submission?

A11. An example (op_wrapper.py) is attached. This wrapper will invoke the executable. Evaluation of hidden test cases using this command: python iccad2026_evaluate.py --evaluate op_wrapper.py